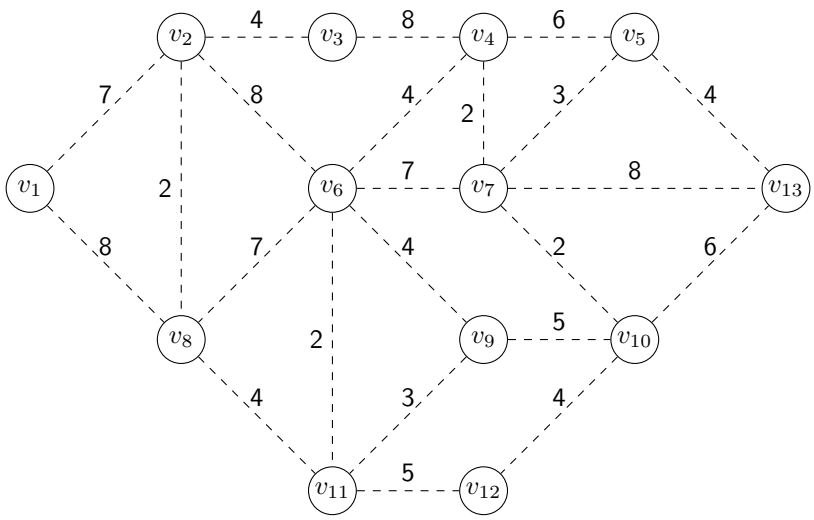
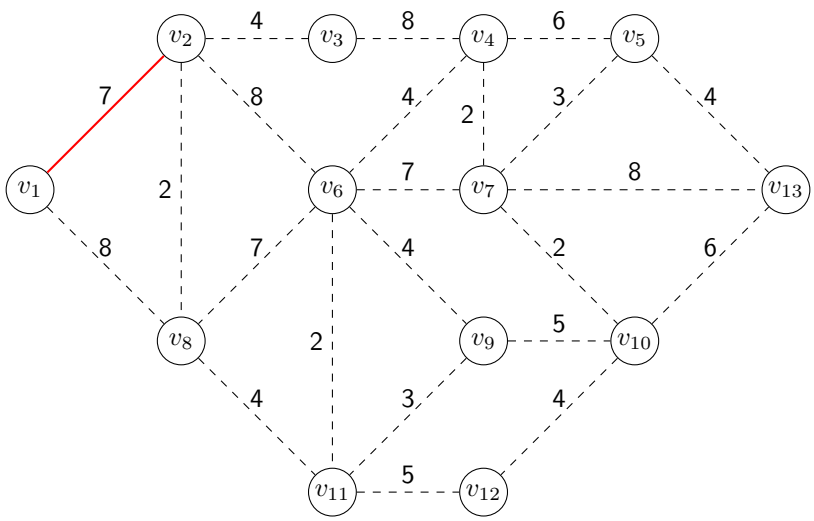
Definitions and introduction

Our next graph algorithm that we will discuss is the shortest path algorithm. For example, what is the shortest path from v_1 to v_{13}?



Perhaps we try and be greedy as we did with our minimum spanning tree algorithm. So, from v_1 we select edge (v_1, v_2) since it has the minimum weight.

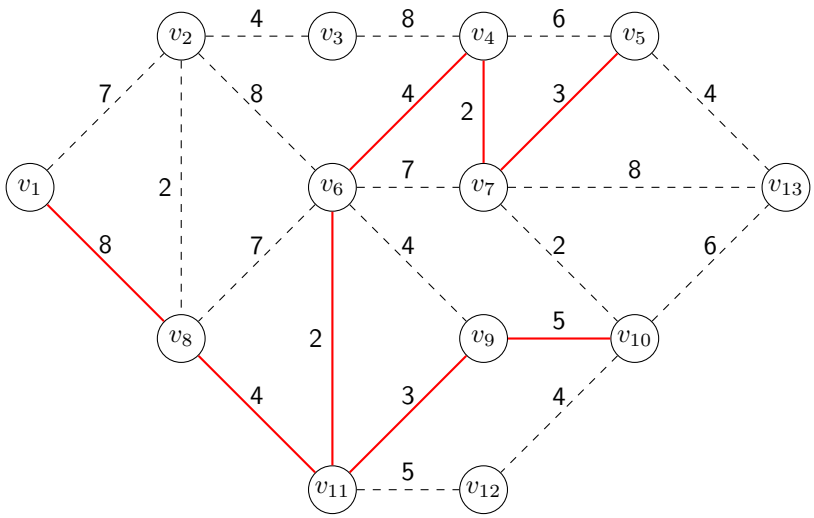


However, from here it is not obvious how we proceed since it seems like we may have to consider every possible path from v_1 to v_{13}. This seems to be a hard obstacle to overcome. So, suppose that you knew the shortest paths to v_5, v_7, and v_{10}. In this case we could exploit the following:

Theorem 1 (Shortest Path Theorem). If $(v_1, v_2, \ldots, v_{i-1}, v_i)$ is a shortest path from v_1 to v_i, then $(v_1, v_2, \ldots, v_{i-1})$ is the shortest path from v_1 to v_{i-1}

Proof. If there is a shorter path P from v_1 to v_{i-1}, then $P \cup (v_{i-1}, v_i)$ would be a shorter path than $(v_1, v_2, \ldots, v_{i-1}, v_i)$. $\square$

Suppose that the following paths illustrated on the graph are the shortest paths to those nodes:



We can compute the distance along these paths. Let $d(u, v) =$ the distance from node u to v, then

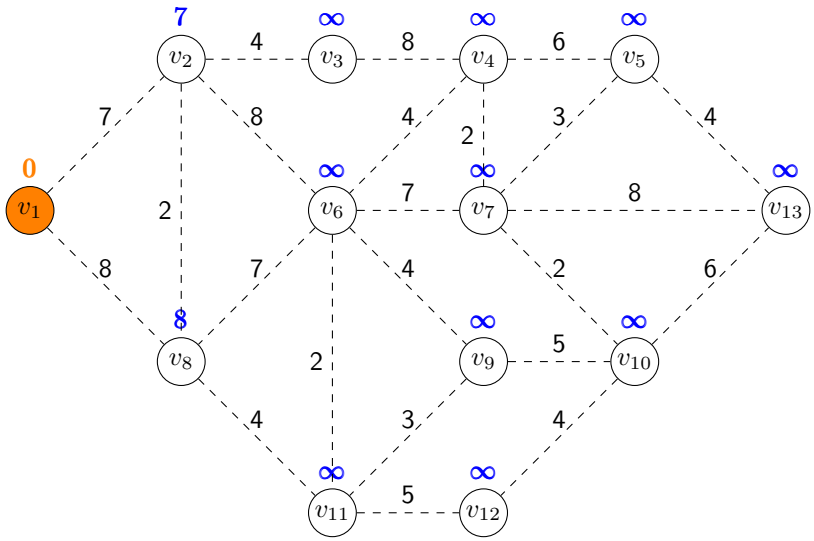
$$d(v_1, v_5) = 23, \qquad d(v_1, v_7) = 20, \qquad d(v_1, v_{10}) = 20.$$

Given these distances, we can see that the shortest path from v_1 to v_{13} that passes through v_5 has length 27; the shortest path from v_1 to v_{13} that passes through v_7 has length 28; the shortest path from v_1 to v_{13} that passes through v_{10} has length 26. Therefore, the shortest path from v_1 to v_{13} is $p = (v_1, v_8, v_{11}, v_9, v_{10}, v_{13})$ and this path has length 26. Having developed this idea that knowing the shortest path to all nodes adjacent to v can inform us about the shortest path to v, we postulate that asking a "harder" question may actual prove be simpler. Namely,

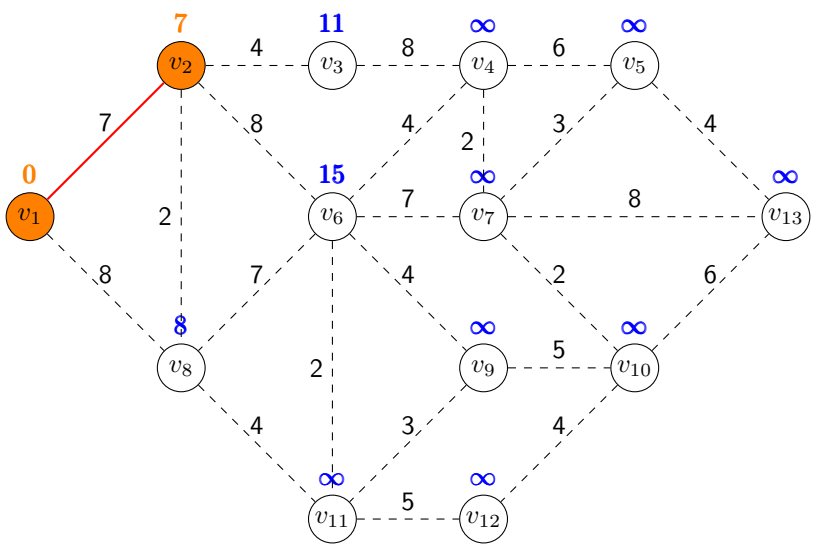
What is the shortest path from v_1 to *every* other node?

We will try and answer this question by building a "shortest path tree" that will store the distance from v_1 to every other node of G. Perhaps we even postulate that a greedy approach to this problem will work. Let us proceed with these assumptions which (if they seem to work) we will prove their correctness.

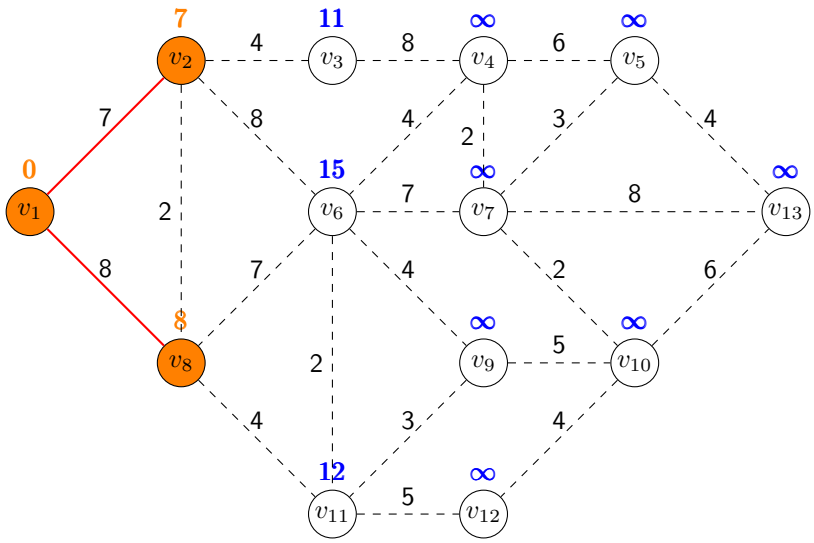
What is the shortest path from v_1 to *every* other node? Motivated by Prim's algorithm, let us begin by initializing the distance to each node to be ∞ (except for v_1 which has a distance 0 and the nodes adjacent to v_1 will have distance equal to the edge weights of the edges incident on the pair of vertices).



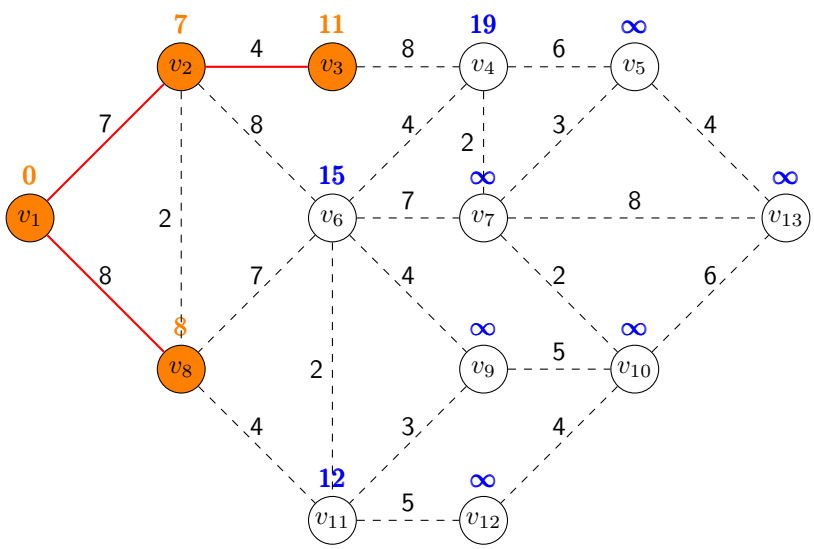
The node that is closest to v_1 will be added to the shortest path tree. The node that is closest to v_1 is v_2 which has a distance of 7. Therefore, we add it to the shortest path tree.



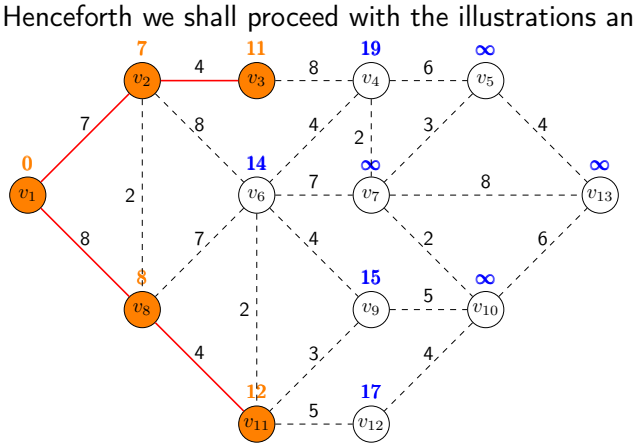
The node that is closest to v_1 will be added to the shortest path tree. The node that is closest to v_1 is v_8 which has a distance of 8. Therefore, we add it to the shortest path tree.

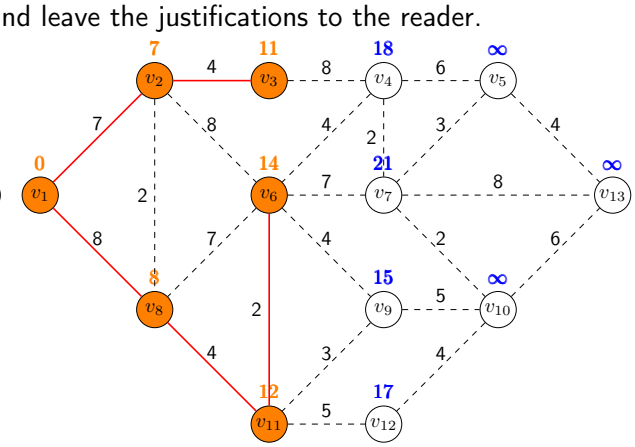


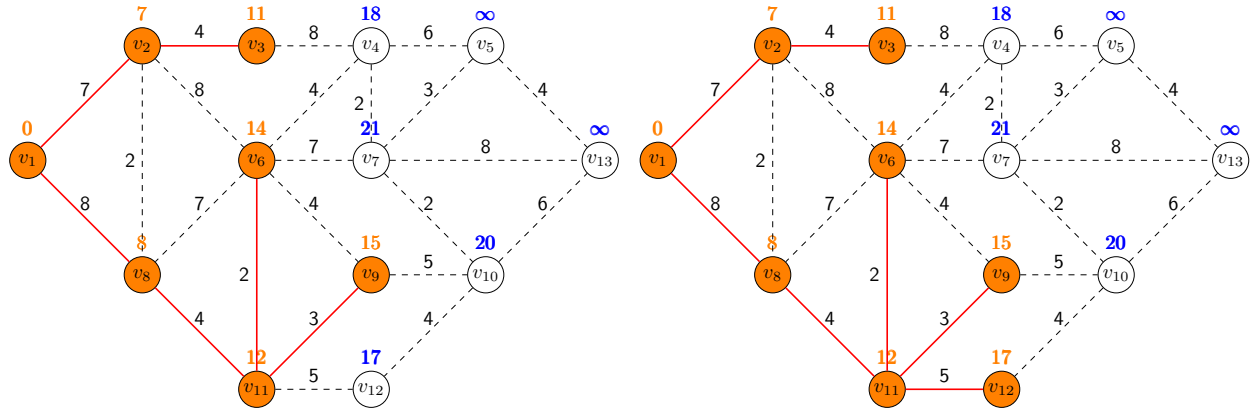
The node that is closest to v_1 will be added to the shortest path tree. The node that is closest to v_1 is v_3 which has a distance of 11. Therefore, we add it to the shortest path tree.

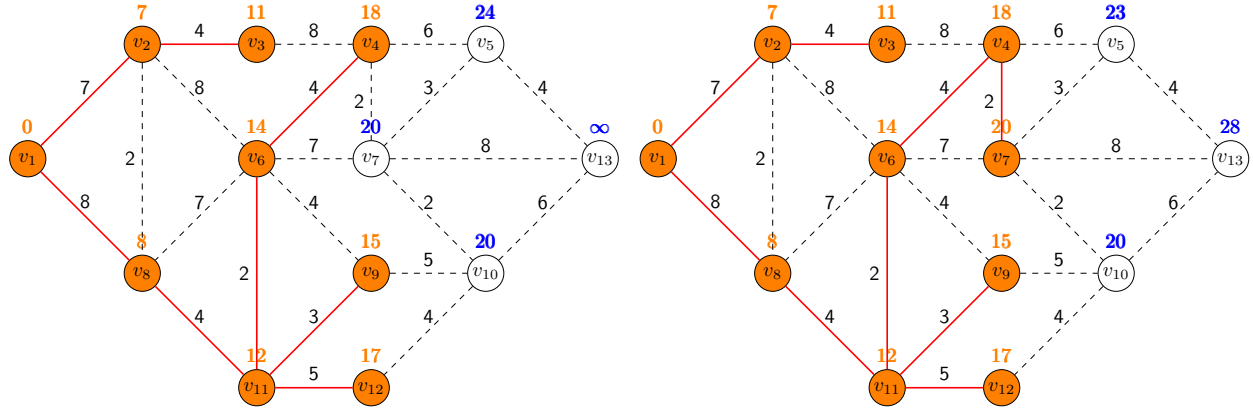


Henceforth we shall proceed with the illustrations and leave the justifications to the reader.



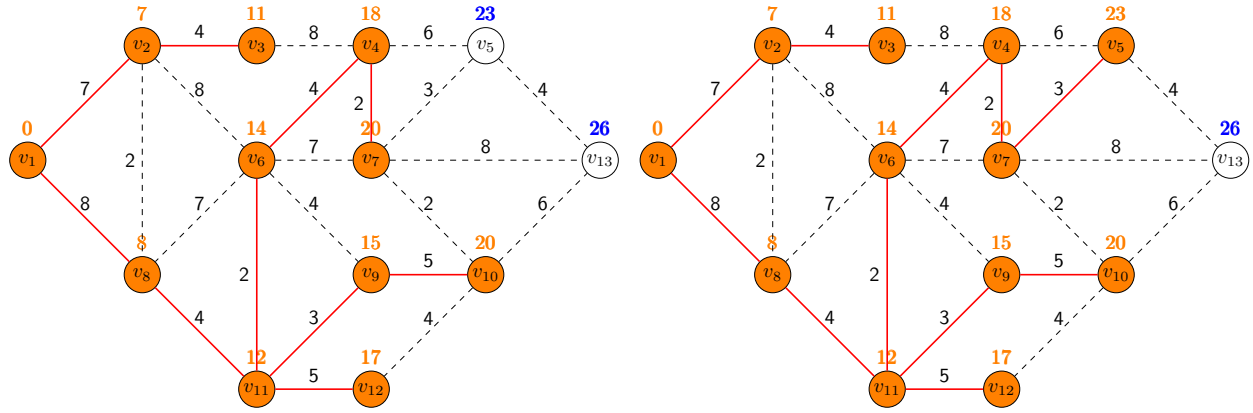



0 7 11 18 ∞
14 21 ∞
$ 15 20
12 17

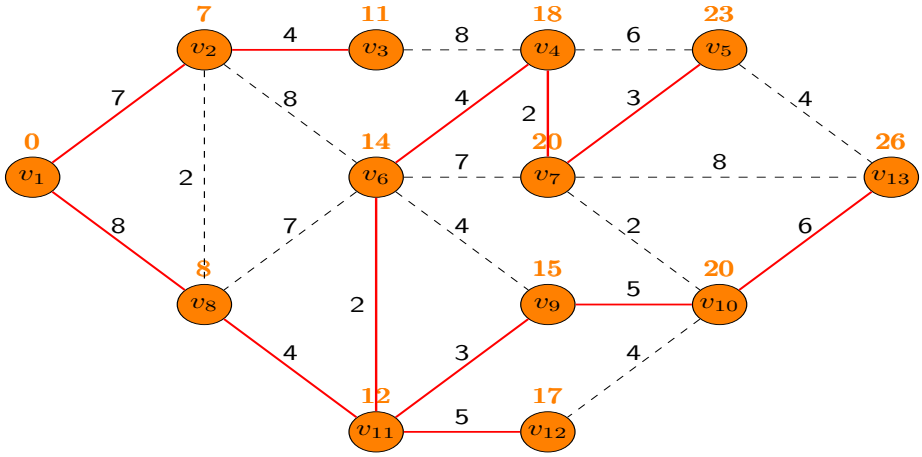

0 7 11 18 ∞
14 21 ∞
$ 15 20
12 17

0 7 11 18 24
14 20 ∞
$ 15 20
12 17

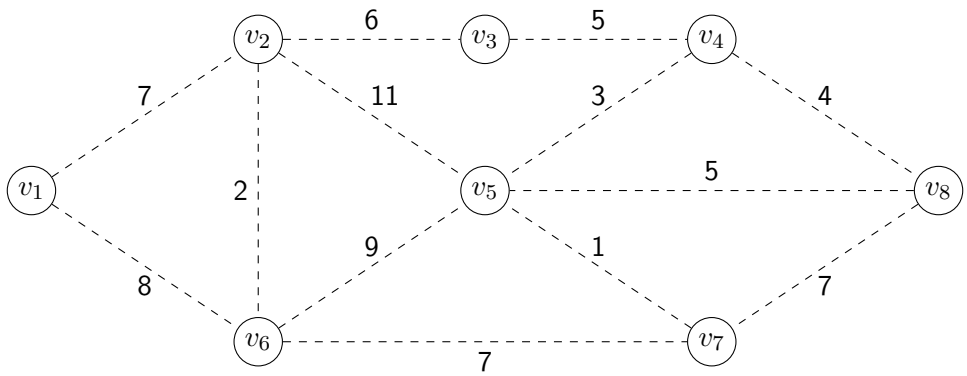
0 7 11 18 23
14 20 28
$ 15 20
12 17


0 7 11 18 23
14 20 26
$ 15 20
12 17

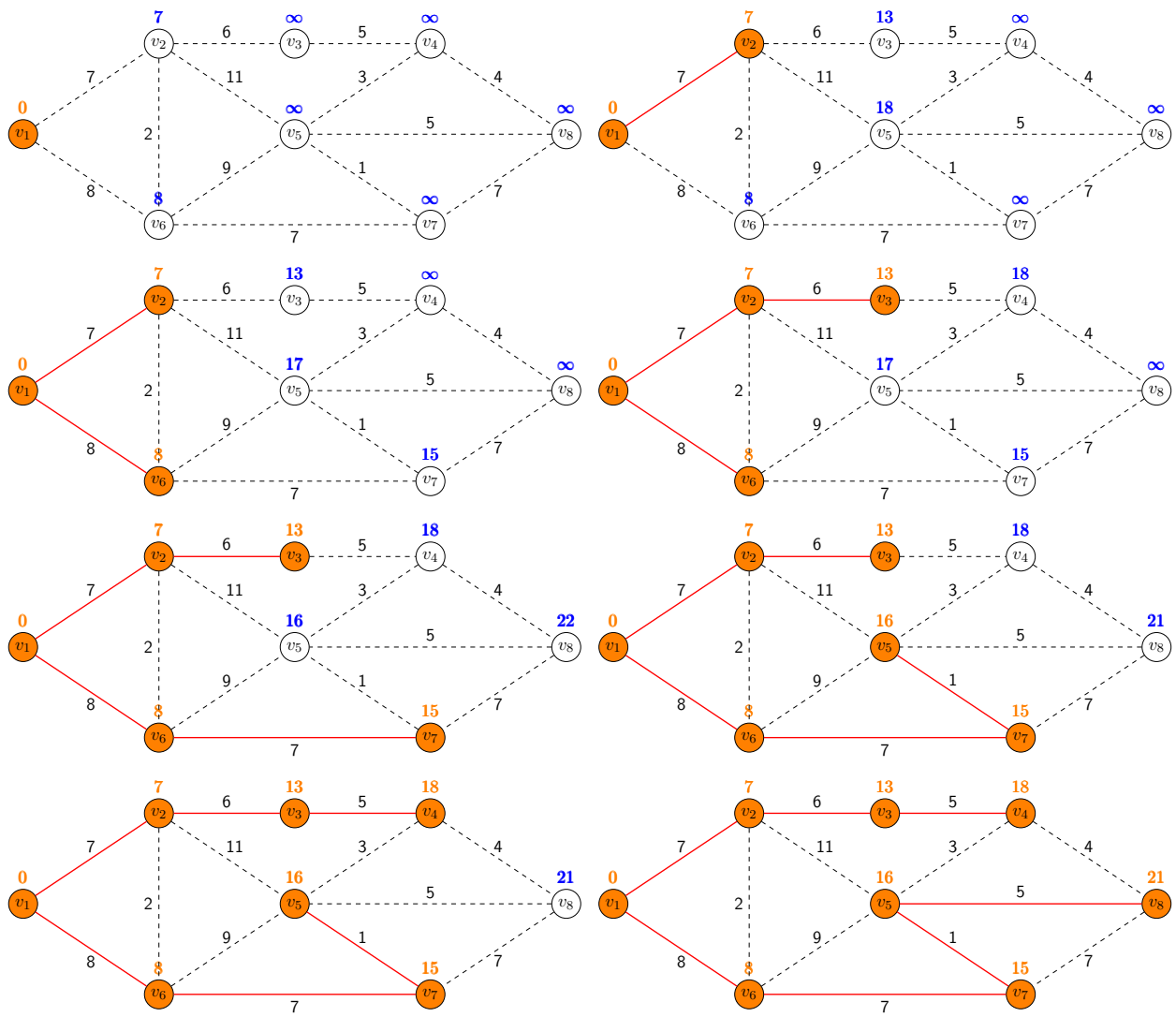
0 7 11 18 23
14 20 26
$ 15 20
12 17


0 7 11 18 23
14 20 26
$ 15 20
12 17

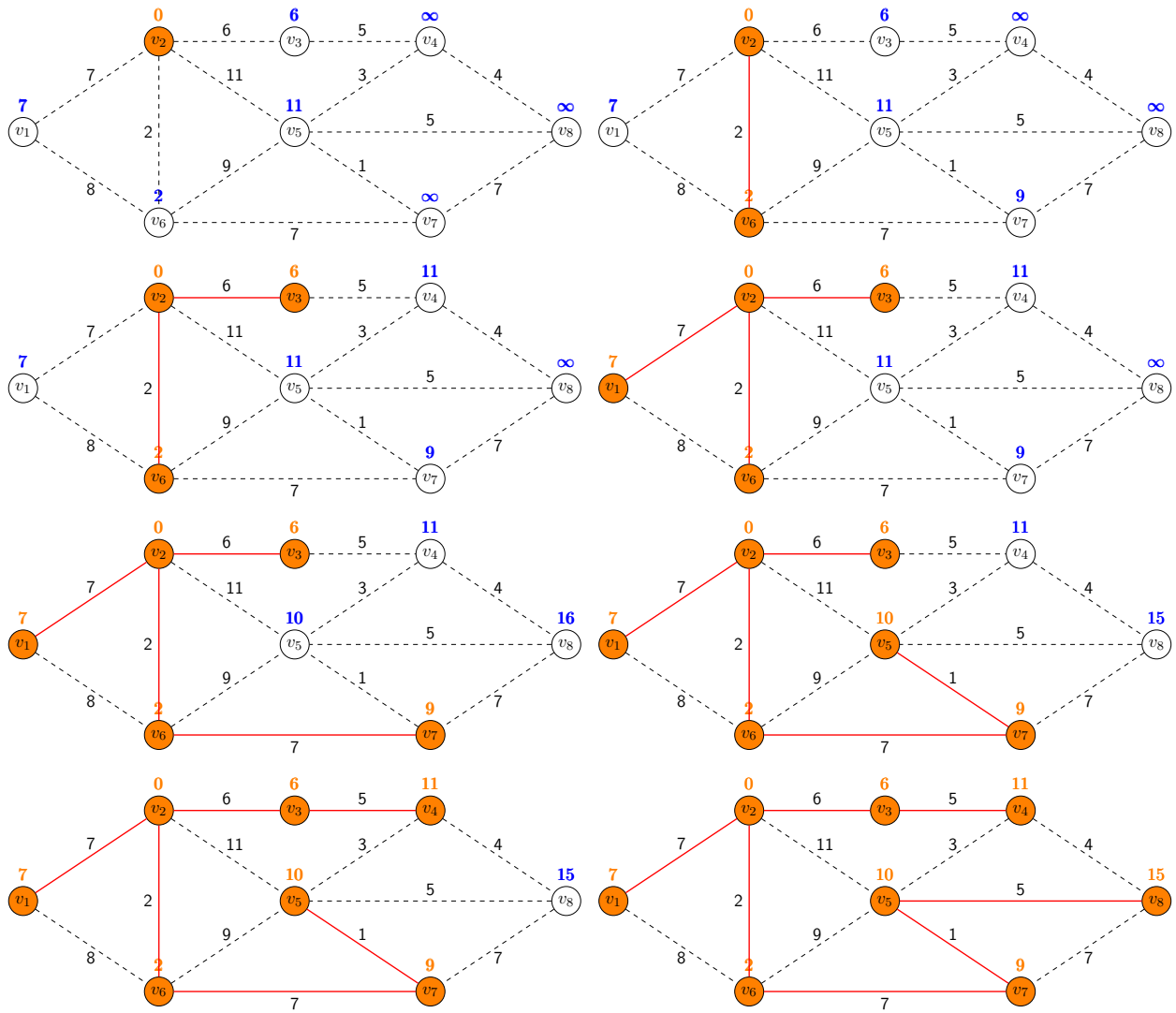
What is the shortest path from v_1 to every other node? What about v_2?



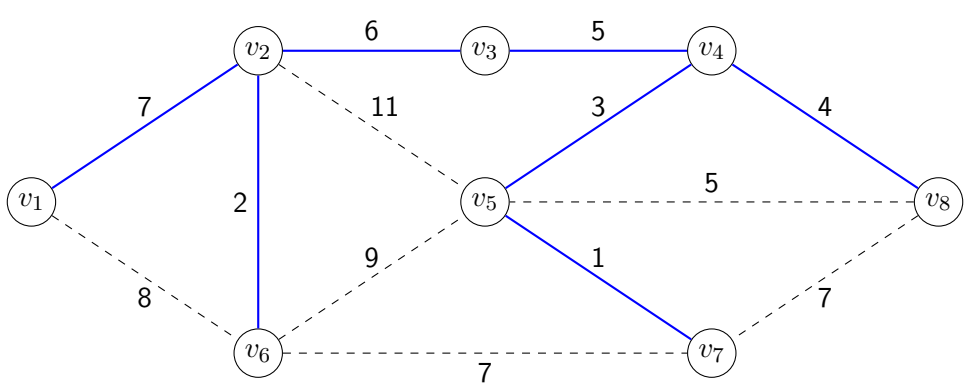
Shortest path from v_1 to ... We begin at v_1 and then iteratively build the shortest path tree



Shortest path from v_2 to ... We begin at v_2 and iteratively build the shortest path tree.



Notice that the shortest path trees are different for the two different nodes; in general this is the case. Consider the **minimum spanning tree** for the graph



and notice that it is distinct from both of the shortest path trees that we have found!

This greedy algorithm for finding the shortest path tree of a given node is known as Dijkstra's algorithm. The code is extremely similar to the code for Prim's Minimum Spanning Tree algorithm; notice that this initial implementation does not compute and store the distances as we have done above. We could modify this code to be more sophisticated and make these computations, but that is not required for the algorithm to function, only for the algorithm to be efficient.

Input: A weighted graph G and starting vertex v_s

Output: Shortest path tree from v_s to all vertices of G

```
function DijkstrasShortestPath(G, v_s)
    U = G.V − {v_s}
    v_s.SPparent = NULL
    while (U ≠ ∅) and (∃ edge from (G.V − U) to U) do
        (v_i, v_j) = edge from G.V − U to U which minimizes v_i.distance + weight(v_i, v_j)
        v_j.SPparent = v_i
        v_j.distance = v_i.distance + weight(v_i, v_j)
        U = U − {v_j}
    end while
end function
```

Let $n =$ number of vertices of G and $m =$ number of edges of G.

Theorem 2 (Shortest Path Theorem 2). Let $d(v_s, v_i) =$ shortest path from v_s to v_i and let U be a subset of $G.V$ where $v_s \notin U$.
If $(v_i, v_j) \in G.E$ minimizes $d(v_s, v_i) + \text{weight}(v_i, v_j)$ among all edges connecting $v_i \in (G.V - U)$ to $v_j \in U$, then

$$d(v_s, v_j) = d(v_s, v_i) + \text{weight}(v_i, v_j) \tag{1}$$

Proof. **Part 1** $d(v_s, v_j) \leq d(v_s, v_i) + \text{weight}(v_i, v_j)$:

Let P be a shortest path from v_s to v_i. By definition, $\text{Length}(P) = d(v_s, v_i)$. Define $P' = P \cup (v_i, v_j)$ which has length $\text{Length}(P') = d(v_s, v_i) + \text{weight}(v_i, v_j)$. Note that P' is *a* path from v_s to v_j and necessarily,

$$d(v_s, v_j) \leq \text{Length}(P') = d(v_s, v_i) + \text{weight}(v_i, v_j),$$

since $d(v_s, v_j)$ is the minimum distance.

Part 2 $d(v_s, v_j) \geq d(v_s, v_i) + \text{weight}(v_i, v_j)$:

Let P be the shortest path from v_s to v_j. Since $v_s \in (G.V - U)$ and $v_j \in U$, path P must contain some edge (v_a, v_b) where $v_a \in (G.V - U)$ and $v_b \in U$. Let P_a be the subpath of P from v_s to v_a. Now,

$$\text{Length}(P) \geq \text{Length}(P_a) + \text{weight}(v_a, v_b) \geq \underbrace{d(v_s, v_a) + \text{weight}(v_a, v_b)}_{\text{by definition of } d},$$

Since (v_i, v_j) is defined to minimize $d(v_s, v_i) + \text{weight}(v_i, v_j)$,

$$d(v_s, v_a) + \text{weight}(v_a, v_b) \geq d(v_s, v_i) + \text{weight}(v_i, v_j).$$

Therefore,

$$\text{Length}(P) \geq d(v_s, v_a) + \text{weight}(v_a, v_b) \geq d(v_s, v_i) + \text{weight}(v_i, v_j)$$

and since P was the shortest path from v_s to v_j,

$$d(v_s, v_j) \geq \text{Length}(P) \geq d(v_s, v_i) + \text{weight}(v_i, v_j)$$

Therefore, since $d(v_s, v_j) \leq d(v_s, v_i) + \text{weight}(v_i, v_j)$ and $d(v_s, v_j) \geq d(v_s, v_i) + \text{weight}(v_i, v_j)$,

$$d(v_s, v_j) = d(v_s, v_i) + \text{weight}(v_i, v_j)$$

where (v_i, v_j) minimizes $d(v_s, v_i) + \text{weight}(v_i, v_j)$ over all $v_i \in (G.V - U)$ to $v_j \in U$. $\square$

We return to the code that we have introduced earlier,

Input: A weighted graph G and starting vertex v_s

Output: Shortest path tree from v_s to all vertices of G

```
1: function DijkstrasShortestPath(G, v_s)
2:     U = G.V − {v_s}
3:     v_s.SPparent = NULL
4:     while (U ≠ ∅) and (∃ edge from (G.V − U) to U) do
5:         (v_i, v_j) = edge from G.V − U to U which minimizes v_i.distance + weight(v_i, v_j)
6:         v_j.SPparent = v_i
7:         v_j.distance = v_i.distance + weight(v_i, v_j)
8:         U = U − {v_j}
9:     end while
10: end function
```

and the clever observer may notice that our theorem does not necessarily directly apply to the code. The theorem claimed we minimized $d(v_s, v_i) + \text{weight}(v_i, v_j)$, but the algorithm minimized $v_i.\texttt{distance} + \text{weight}(v_i, v_j)$, therefore we must show that

Proposition 1. At each iteration, $v_k.\texttt{distance} = d(v_s, v_k)$ for every $v_k \in (G.V - U)$.

Proof. Let U_p be U at the beginning of the pth iteration. We prove it by induction on p. First, we prove the base case of $G.V - U_1 = \{v_s\}$ with $\text{distance}(v_s, v_s) = 0 = d(v_s, v_s)$.
Suppose that $v_k.\texttt{distance} = d(v_s, v_k)$ for every $v_k \in (G.V - U_p)$. We must show that $v_k.\texttt{distance} = d(v_s, v_k)$ for every $v_k \in (G.V - U_{p+1})$.
According to line 8 in `DijkstrasShortestPath`, $(G.V - U_{p+1})$ has one more vertex than $(G.V - U_p)$. Further, line 5 chooses (v_i, v_j) such that it is the edge from $G.V - U_p$ to U_p which minimizes $v_i.\texttt{distance} + \text{weight}(v_i, v_j)$. By inductive assumption (since $v_i \in (G.V - U_p)$), $v_i.\texttt{distance} = d(v_s, v_i)$, so edge (v_i, v_j) minimizes $d(v_s, v_i) + \text{weight}(v_i, v_j)$.
By the **Shortest Path Theorem 2**, $d(v_s, v_j) = d(v_s, v_i) + \text{weight}(v_i, v_j)$. As we have mentioned above, $v_i.\texttt{distance} = d(v_s, v_i)$, so (from line 7)

$$v_j.\texttt{distance} = v_i.\texttt{distance} + \text{weight}(v_i, v_j) = d(v_s, v_i) + \text{weight}(v_i, v_j)$$

and $G.V - U_{p+1} = (G.V - U_p) \cup \{v_j\}$. Thus the proposition holds for $p + 1$ and the result follows by mathematical induction. □

Having finally proven the correctness of our algorithm, we can begin analyzing the running time of the algorithm. The running time of this algorithm is *identical* to the running time of Prim's MST algorithm since the only meaningful difference is the quantity that is being minimized. Therefore, the running time of this implementation is $\boxed{\Theta(n + nm)}$. We have included a more formal argument on the following page for those that are interested in seeing it replicated.

Let $n =$ number of verticies of G and $m =$ number of edges of G. We shall assume that G is a connected graph to simplify our analysis.

Line 2 of the code takes $\Theta(n)$ time to create the set U of size $n-1$.

Line 3 of the code takes $\Theta(1)$ time.

Since G is a connected graph, the while loop on lines 4-9 will run approximately n times to add each vertex to the spanning tree. We note that line 4 may not take constant time to execute since we may not be able to determine whether an edge exists from $G.V - U$ to U in constant time. In the worst case this could take cm time since we may have to check every edge of the graph.

The seemingly innocuous line 5 may not take constant time since in order to determine the minimum weight edge from $G.V - U$ we may have to examine every edge in the graph (or at the very least every edge in U or $G.V - U$). We shall assume that line 5 takes $\Theta(m)$ time to find the edge.

Lines 6 and 7 both take $\Theta(1)$ time.

Therefore, the running time of `DijkstrasShortestPath` is in $\boxed{\Theta(n + mn)}$. We note that $n - 1 \leq m \leq \frac{n(n-1)}{2}$ for a connected graph, so `DijkstrasShortestPath` is in $\Omega(n^2)$ and $O(n^3)$, which are both rather unfortunate bounds.

Just as was the case for Prim's algorithm, we can make this algorithm more efficient by increasing the storage requirements. If we store the "current best" distance at each node (just as we stored the "current best edge" for the MST), then we can meaningfully improve the running time.

Input: A weighted graph G and starting vertex v_s

Output: Shortest path tree from v_s to all vertices of G

```
function DijkstraShortestPath(G, v_s)
    U = G.V
    for each vertex v_i ∈ G.V − {v_s} do
        v_i.distance = ∞
    end for
    v_s.distance = 0
    v_s.parent = NULL
    while (U ≠ ∅) and (v_i.distance < ∞ for some v_i ∈ U) do
        v_j = v_i ∈ U with minimum v_i.distance
        U = U − {v_j}
        v_j.SPparent = v_j.parent
        for each edge (v_j, v_k) incident on v_j do
            newDist = v_j.distance + weight(v_j, v_k)
            if (v_k ∈ U) and (newDist < v_k.distance) then
                v_k.parent = v_j
                v_k.distance = newDist
            end if
        end for
    end while
end function
```

Let $n =$ number of vertices of G and $m =$ number of edges of G. Just like our first implementation, the running time of this algorithm is *identical* to the running time of Prim's MST; therefore the running time of this algorithm is $\boxed{\Theta(n^2 + m)}$. We have included a more formal argument on the following page for those that are interested in seeing it replicated. Now, we might again realize that we could utilize a min heap to make the algorithm potentially even more efficient.

Let $n =$ number of verticies of G and $m =$ number of edges of G. We shall assume that G is a connected graph to simplify our analysis.
Lines 3-5 take $\Theta(n)$ time since we must iterate over $n - 1$ vertices.
Lines 6 and 7 take $\Theta(1)$ time.
The while loop on lines 8-19 iterates n times.
Line 9 must iterate over every vertex in U to determine the minimum and therefore takes $\Theta(|U|)$ time. Notice that as the while loop iterates $|U|$ decreases from n down to 1. Therefore, the cost line 9 on the ith iteration is $c(n - i + 1)$. Therefore, the total cost of line 9 is

$$\text{Cost of line 9} = \sum_{i=1}^{n} c(n - i + 1) = \sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

Lines 10 and 11 take $\Theta(1)$ time.
The for loop on lines 12-18 iterates deg(v_j) times and each iteration takes constant time (note that determining whether $v_k \in U$ in constant time may require some cautious programming). Therefore,

$$\text{Cost of lines 12-18} = \sum_{j=1}^{n} c\text{deg}(v_j) = 2cm$$

Therefore, the total cost of the while loop on lines 8-19 is $\Theta(n^2 + m)$. Furthermore, the running time of the algorithm is

$$\boxed{T(m, n) \in \Theta(n^2 + m)}$$

A quick refresher on a Minimum Priority Queue,

Method	**Functionality**	**Runtime**
`Q.Init()`	Initialize the minimum priority queue	$\Theta(1)$
`Q.Insert`(Object x, Key k)	Inserts x into Min. PQ with key k	$\Theta(\log(s))$
`Q.ExtractMin()`	Extracts element with minimum key	$\Theta(\log(s))$
`Q.DecreaseKey`(Object x, Key k)	Decrease the key value associated with x to k	$\Theta(\log(s))$
`Q.IsEmpty()`	Returns true if Min. PQ is empty.	$\Theta(1)$
`Q.IsNotEmpty()`	Returns true if Min. PQ is not empty.	$\Theta(1)$
`Q.Contains`(Object x)	Returns true if Min. PQ contains x.	$\Theta(1)$
`Q.Key`(Object x)	Returns key value associated with x.	$\Theta(1)$
`Q.MinKey()`	Returns minimum key value.	$\Theta(1)$

Note: We will need to add an array of size n (the number of vertices in the graph) to the Minimum Priority Queue that indicates the location of each vertex in the heap.

Input: A weighted graph G and starting vertex v_s

Output: Shortest path tree from v_s to all vertices of G

```
function DijkstraShortestPath(G, v_s)
    Q.Init()
    for each vertex v_i ∈ G.V − {v_s} do
        Q.Insert(v_i, ∞)
    end for
    Q.Insert(v_s, 0)
    v_s.parent = NULL
    while (Q.IsNotEmpty()) and (Q.MinKey() ≠ ∞) do
        minDist = Q.MinKey()
        v_j = Q.ExtractMin()
        v_j.SPparent = v_j.parent
        v_j.distance = minDist
        for each edge (v_j, v_k) incident on v_j do
            newDist = v_j.distance + weight(v_j, v_k)
            if (Q.Contains(v_k)) and (newDist < Q.Key(v_k)) then
                v_k.parent = v_j
                Q.DecreaseKey(v_k, newDist)
            end if
        end for
    end while
end function
```

Let n = number of vertices of G and m = number of edges of G. Just like our first and second implementations, the running time of this algorithm is *identical* to the running time of Prim's MST; therefore the running time of this algorithm is $\boxed{O(n\log(n) + m\log(n))}$. We have included a more formal argument on the following page for those that are interested in seeing it replicated.

Let $n =$ number of verticies of G and $m =$ number of edges of G. We shall assume that G is a connected graph to simplify our analysis.
The running time of lines 3-7 is

$$\sum_{s=1}^{n} T_{in}(s) = \sum_{s=1}^{n} c\log(s) \in \Theta(n\log(n))$$

since the running time of `Q.Insert` is $\Theta(\log(s))$ where s is the size of the heap.
The while loop on lines 8-20 runs n times.
The size before extraction of the heap on line 10 starts at n and decreases down to 1, so

$$\text{Cost of line 10} = \sum_{s=1}^{n} T_{em}(s) = \sum_{s=1}^{n} c\log(s) \in \Theta(n\log(n))$$

The running time of lines 11 and 12 is $\Theta(1)$.
The for loop on lines 13-19 runs deg(v_j) times and each iteration of the loop takes $\Theta(\log(s))$ time, where s is the number of elements in the heap.
Let s_j be the size of the heap when v_j is extracted from the heap, then

$$\text{Cost of lines 13-19} = \sum_{j=1}^{n} c\text{deg}(v_j)\log(s_j) \leq \sum_{s=1}^{n} c\text{deg}(v_j)c\log(n) = 2cm\log(n)$$

Bounding this summation from below proves too difficult for this course and requires more assumptions than we are willing to make here.
Therefore,

$$\boxed{T(m,n) \in O(n\log(n) + m\log(n))}$$

which at first glance looks like an improvement over our running time from before, but if G is the complete graph and $m = \frac{n(n-1)}{2}$, then this implementation takes $\Theta(n^2\log(n))$ time while the alternative implementation takes $\Theta(n^2)$ time.